

Lembar Materi & Praktikum: Real-Time API dengan Server-Sent Events (SSE) dan WebSockets pada Golang Gin

Durasi: 3 Jam (180 Menit) **Level:** Advanced (Lanjut)

Praktikum ini melengkapi materi teori pada [22. real-time-api.md](#). Anda akan dipandu untuk menerapkan komunikasi data seketika (*real-time communication*) pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) dengan framework **Gin**. Kita akan belajar membandingkan keterbatasan kueri REST HTTP konvensional, merancang aliran data satu arah terusan menggunakan **Server-Sent Events (SSE)** untuk monitoring sisa kuota tiket saat *war*, serta membangun server komunikasi dua arah interaktif menggunakan **WebSockets** dan pustaka **Gorilla WebSocket**.

Sesi 1: Teori Keterbatasan REST & Konsep Komunikasi Real-Time (45 Menit)

1. Mengapa HTTP REST Konvensional Tidak Efektif untuk Aplikasi Real-Time?

Protokol HTTP/1.1 standar didesain menggunakan pola **Request-Response (Stateless)**. Klien mengirim permintaan, server memproses dan membalas, kemudian koneksi TCP segera ditutup. Pola ini menyulitkan sistem yang membutuhkan pembaruan data secara langsung (*live updates*).

Pendekatan Tradisional (Tidak Disarankan):

- **Short Polling:** Klien menembak API backend (misal: `GET /api/v1/concerts/1/quota`) berulang kali setiap 2 detik.
 - *Kelemahan:* Sangat memboroskan kuota internet klien, membebani database backend dengan kueri berulang, dan menghabiskan sumber daya I/O server.
- **Long Polling:** Klien mengirim request, server menahan koneksi tetap terbuka (*hanging connection*) sampai ada data baru, lalu membalas. Klien segera mengirim request baru setelah respons diterima.
 - *Kelemahan:* Tetap memakan resource koneksi server, dan memiliki delay jeda saat membuat koneksi baru.

2. Solusi Modern: SSE vs WebSockets

Untuk menanggulangi pemborosan kueri di atas, industri backend menggunakan dua metode real-time utama:

Kriteria	Server-Sent Events (SSE)	WebSockets (WS)
Protokol	HTTP standar (<code>text/event-stream</code>)	Protokol khusus TCP (<code>ws://</code> atau <code>wss://</code>)
Arah Komunikasi	Satu arah (<i>Unidirectional</i> - Server ke Klien)	Dua arah (<i>Bidirectional / Full-Duplex</i>)
Kemudahan Konfigurasi	Sangat mudah (tidak butuh library tambahan)	Membutuhkan library Upgrader (Gorilla WS)
Auto-Reconnect	Didukung secara otomatis oleh browser	Harus ditulis secara manual di frontend
Kasus Penggunaan	Live stock kuota tiket, live notification feeds.	Chatting, game multiplayer, kolaborasi dokumen.

Sesi 2: Teori Server-Sent Events (SSE) & WebSocket Handshake (45 Menit)

1. Mekanisme Server-Sent Events (SSE)

SSE memanfaatkan koneksi HTTP persisten (*persistent HTTP connection*). Server mengirim respons dengan header khusus:

- `Content-Type: text/event-stream` : Menginstruksikan browser agar tidak menutup koneksi dan memperlakukan setiap baris data sebagai event stream.
- `Cache-Control: no-cache` : Mencegah browser menyimpan cache respons.
- `Connection: keep-alive` : Menjaga koneksi socket TCP tetap terbuka.

Di dalam Gin, kita menggunakan method `c.Stream(func(w io.Writer) bool { ... })` yang mengeksekusi loop pengiriman data secara terus-menerus tanpa memutus respon HTTP.

2. Mekanisme WebSocket Handshake (HTTP Upgrade)

Protokol WebSocket diawali dengan permintaan HTTP GET biasa dari klien yang menyertakan header permintaan peningkatan protokol:

```
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Key: dGhLIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Server yang mendukung WebSocket akan memverifikasi dan membalas dengan status **HTTP 101 Switching Protocols**. Setelah jabat tangan (*handshake*) ini sukses, koneksi HTTP "dihancurkan" dan digantikan oleh koneksi dua arah persisten `ws://` langsung di atas socket TCP.

Di dalam Go, kita memanfaatkan pustaka **Gorilla WebSocket** (github.com/gorilla/websocket) yang menyediakan objek `websocket.Upgrader` untuk menangani proses perubahan protokol ini secara otomatis.

Sesi 3: Laboratorium Praktikum Mandiri – Real-Time Ticket Quota Stream dengan SSE (45 Menit)

Pelajari langkah-langkah pembuatan streaming data searah menggunakan Gin SSE di bawah ini.

1. Membuat Endpoint SSE di Gin: [handler/concert_sse.go](#)

Buat berkas baru bernama `concert_sse.go` di bawah folder `handler/` untuk memancarkan perubahan kuota tiket secara real-time:

```
package handler

import (
    "fmt"
    "io"
    "net/http"
    "time"

    "go-tiket-konser/service"

    "github.com/gin-gonic/gin"
)

type ConcertSSEHandler struct {
    ticketService service.TicketCategoryService // Membutuhkan service kuota tiket
}
```

```

func NewConcertSSEHandler(ts service.TicketCategoryService) *ConcertSSEHandler {
    return &ConcertSSEHandler{ticketService: ts}
}

// StreamTicketQuota menyemburkan data kuota tiket sisa secara live melalui SSE
func (h *ConcertSSEHandler) StreamTicketQuota(c *gin.Context) {
    idStr := c.Param("id") // ID konser

    // 1. Atur Header HTTP Wajib untuk SSE
    c.Writer.Header().Set("Content-Type", "text/event-stream")
    c.Writer.Header().Set("Cache-Control", "no-cache")
    c.Writer.Header().Set("Connection", "keep-alive")
    c.Writer.Header().Set("Transfer-Encoding", "chunked")

    // 2. Gunakan c.Stream untuk loop pengiriman data tanpa memutus koneksi
    c.Stream(func(w io.Writer) bool {
        // Konversi string ID konser ke int
        var concertID int
        fmt.Sscanf(idStr, "%d", &concertID)

        // Ambil data kategori tiket dari database/service
        categories, err := h.ticketService.GetCategoriesByConcertID(concertID)
        if err != nil {
            // Jika terjadi error, kirim event error dan hentikan stream
            c.SSEvent("error", "Gagal memuat kuota tiket")
            return false // Mengembalikan false akan memutus koneksi
streaming
        }

        // Bungkus informasi sisa kuota ke dalam format string sederhana atau
JSON
        var quotaData string
        for _, cat := range categories {
            quotaData += fmt.Sprintf("[%s: Tersisa %d/%d] ", cat.Name,
cat.AvailableQuota, cat.TotalQuota)
        }

        // 3. Pancarkan event dengan nama "quota_update" ke klien
        c.SSEvent("quota_update", quotaData)

        // Jeda kueri selama 3 detik sebelum melakukan kueri ulang
        time.Sleep(3 * time.Second)

        return true // Mengembalikan true akan melanjutkan kueri pada
perulangan berikutnya
    })
}

```

Daftarkan rute ini pada file `routes/routes.go` :

```
// GET /api/v1/concerts/:id/stream-quota - Diakses tanpa proteksi token untuk umum
api.GET("/concerts/:id/stream-quota", concertSSEHandler.StreamTicketQuota)
```

Sesi 4: Laboratorium Praktikum Mandiri – Chat Room Kolaboratif dengan WebSockets (45 Menit)

Kita akan merancang server WebSocket yang aman dari kebocoran memori dan aman dari *race condition* (menggunakan locking mutex).

1. Membuat WebSocket Handler & Connection Manager: `handler/chat_ws.go`

Pertama, pastikan pustaka gorilla websocket terinstal di terminal proyek Anda:

```
go get -u github.com/gorilla/websocket
```

Buat berkas baru bernama `chat_ws.go` di bawah folder `handler/` untuk menangani chat antar pelanggan atau admin:

```
package handler

import (
    "log"
    "net/http"
    "sync"

    "github.com/gin-gonic/gin"
    "github.com/gorilla/websocket"
)

// 1. Inisialisasi WebSocket Upgrader
var upgrader = websocket.Upgrader{
    ReadBufferSize: 1024,
    WriteBufferSize: 1024,
    CheckOrigin: func(r *http.Request) bool {
        return true // Izinkan lintas origin (CORS Bypass) demi kemudahan
        pengujian
    },
}

// 2. Hub Connection Manager (Thread-Safe dengan sync.RWMutex)
type ChatHub struct {
    clients map[*websocket.Conn]bool
}
```

```

    mu    sync.RWMutex
}

func NewChatHub() *ChatHub {
    return &ChatHub{
        clients: make(map[*websocket.Conn]bool),
    }
}

// Register mendaftarkan koneksi klien baru
func (h *ChatHub) Register(conn *websocket.Conn) {
    h.mu.Lock()
    defer h.mu.Unlock()
    h.clients[conn] = true
    log.Printf("Klien terhubung baru. Total aktif: %d", len(h.clients))
}

// Unregister menghapus koneksi klien yang terputus
func (h *ChatHub) Unregister(conn *websocket.Conn) {
    h.mu.Lock()
    defer h.mu.Unlock()
    if _, exists := h.clients[conn]; exists {
        conn.Close()
        delete(h.clients, conn)
        log.Printf("Klien keluar. Total aktif: %d", len(h.clients))
    }
}

// Broadcast mengirim pesan ke seluruh klien aktif
func (h *ChatHub) Broadcast(messageType int, message []byte) {
    h.mu.Lock()
    defer h.mu.Unlock()

    for client := range h.clients {
        err := client.WriteMessage(messageType, message)
        if err != nil {
            log.Println("Gagal mengirim pesan ke satu klien, menutup koneksi...")
            client.Close()
            delete(h.clients, client)
        }
    }
}

type ChatWSHandler struct {
    Hub *ChatHub
}

func NewChatWSHandler(hub *ChatHub) *ChatWSHandler {
    return &ChatWSHandler{Hub: hub}
}

// HandleChatWS memproses jabat tangan HTTP Upgrade ke WS dan mengelola loop baca

```

```

pesan
func (h *ChatWSHandler) HandleChatWS(c *gin.Context) {
    // a. Upgrade koneksi HTTP biasa menjadi WebSocket persistent TCP
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    if err != nil {
        log.Println("Gagal melakukan upgrade koneksi:", err)
        return
    }
    // Pastikan koneksi dibersihkan saat fungsi keluar
    defer h.Hub.Unregister(conn)

    // b. Daftarkan koneksi ke Hub manager
    h.Hub.Register(conn)

    // c. Infinite loop untuk membaca pesan yang dikirim oleh klien ini
    for {
        msgType, msg, err := conn.ReadMessage()
        if err != nil {
            // Loop terputus jika klien menutup browser atau koneksi
            log.Println("Koneksi ditutup oleh klien:", err)
            break
        }

        log.Printf("Pesan diterima: %s", string(msg))

        // d. Sebarkan pesan ke seluruh klien yang terhubung (Broadcasting)
        h.Hub.Broadcast(msgType, msg)
    }
}

```

Daftarkan rute ini pada file `routes/routes.go` :

```

// Inisialisasi Hub
chatHub := handler.NewChatHub()
chatWSHandler := handler.NewChatWSHandler(chatHub)

// GET /api/v1/chat - Sambungan WebSocket
api.GET("/chat", chatWSHandler.HandleChatWS)

```

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan arsitektur real-time ini secara mandiri untuk meningkatkan pemahaman Anda tentang otorisasi dan sistem terdistribusi.

Tantangan 1: Autentikasi JWT pada Sambungan WebSocket

Handshake WebSocket diawali dengan permintaan HTTP GET biasa. Kita tidak bisa mengirim header kustom `Authorization: Bearer <token>` melalui objek WebSocket JavaScript browser bawaan secara mudah. Cara standar industri adalah mengirimkan token JWT melalui **URL Query Parameter**.

- **Tugas:**

1. Ubah handler `HandleChatWS` di `handler/chat_ws.go`.
2. Sebelum memanggil `upgrader.Upgrade()`, tangkap parameter kueri token dari URL: `c.Query("token")`.
3. Panggil fungsi validasi JWT yang telah Anda buat di modul sebelumnya untuk memeriksa keabsahan token.
4. Jika token kosong atau tidak valid, batalkan proses upgrade dan kembalikan status HTTP 401 Unauthorized:

```
c.JSON(http.StatusUnauthorized, gin.H{"error": "Handshake ditolak: Token tidak valid"})
return
```

5. Uji coba koneksi WebSocket di Postman dengan URL:

```
ws://localhost:8080/api/v1/chat?token=<JWT_TOKEN_VALID>
```

Tantangan 2: Broadcaster Berbasis Redis Pub/Sub untuk Multi-Instance Server

Hub manager `ChatHub` menyimpan data koneksi klien di memori RAM satu server (`h.clients map[*websocket.Conn]bool`). Jika aplikasi dideploy dalam 2 instance kontainer di balik Load Balancer, klien yang terhubung ke Server 1 tidak akan bisa bertukar pesan dengan klien di Server 2 karena memori RAM-nya terisolasi.

- **Tugas:**

1. Gunakan Redis Client (`config.RedisClient`) untuk menjembatani pesan antar server menggunakan fitur **Redis Pub/Sub** (Publish/Subscribe).
2. Setiap kali server backend menyala, jalankan goroutine yang berlangganan (*subscribe*) ke channel Redis bernama `"chat_channel"`:

```
pubsub := config.RedisClient.Subscribe(ctx, "chat_channel")
// Loop mendengarkan pesan dari Redis Pub/Sub
go func() {
    ch := pubsub.Channel()
    for msg := range ch {
        // Begitu menerima pesan dari Redis, sebarkan ke klien lokal di
        server ini
        chatHub.Broadcast(websocket.TextMessage, []byte(msg.Payload))
    }
}
```

```
}  
}()
```

3. Ubah logika saat menerima pesan dari klien di handler. Alih-alih memanggil `h.Hub.Broadcast()` secara lokal langsung, kirimkan pesan tersebut ke channel Redis (*Publish*):

```
config.RedisClient.Publish(ctx, "chat_channel", string(msg))
```

4. Arsitektur ini menjamin seluruh server backend tersinkronisasi secara real-time meskipun aplikasi dideploy dalam puluhan instance kontainer terpisah.